

Sistema de Detección de Contaminación Mediante Procesamiento de Imágenes e Inteligencia Artificial

Cristopher Montero Campos
Universidad de Costa Rica, San José, Costa Rica

Resumen—Este proyecto presenta el desarrollo de un sistema para la detección de contaminación en muestras de arroz mediante técnicas de procesamiento digital de imágenes y modelos clásicos de inteligencia artificial. El flujo de trabajo inicia con el preprocesamiento de imágenes originales, donde se realiza la conversión a escala de grises y el redimensionamiento a una resolución uniforme de 128×128 píxeles. Posteriormente, las imágenes son binarizadas para representar cada píxel mediante valores binarios, permitiendo construir matrices numéricas aptas para análisis computacional.

Una vez obtenidas las matrices binarias, estas se transforman en vectores de características de 16,384 elementos, los cuales conforman el dataset utilizado para el entrenamiento y evaluación de distintos modelos de clasificación. Cada muestra es etiquetada según la presencia o ausencia de contaminación, permitiendo abordar el problema como una clasificación binaria supervisada.

Para la etapa de aprendizaje automático se implementaron y compararon diferentes algoritmos de clasificación: Árbol de Decisión, Naive Bayes, K-Nearest Neighbors (KNN) y Support Vector Machine (SVM). Cada modelo fue entrenado y evaluado utilizando métricas como accuracy, precision y recall, con el objetivo de determinar cuál ofrece el mejor desempeño en la detección de contaminación.

Los resultados obtenidos muestran que el modelo SVM presentó el mejor rendimiento general, alcanzando mayores valores de exactitud y capacidad de clasificación frente a los demás algoritmos evaluados.

I. INTRODUCCIÓN

La detección automática de contaminación en productos agrícolas representa un problema de gran importancia en aplicaciones industriales y de control de calidad. Tradicionalmente, este proceso se realiza de manera manual mediante inspección visual, lo cual puede resultar lento, subjetivo y propenso a errores humanos. Debido a esto, el uso de técnicas de visión por computadora e inteligencia artificial ha adquirido relevancia como alternativa para automatizar este tipo de tareas.

En este proyecto se desarrolla un pipeline completo de procesamiento de imágenes orientado a la detección de contaminación en muestras de arroz. El sistema transforma imágenes originales en representaciones binarias simplificadas, permitiendo extraer información relevante de manera eficiente. Posteriormente, dichas representaciones son utilizadas para entrenar modelos de clasificación supervisada capaces de distinguir entre muestras contaminadas y no contaminadas.

El proyecto incluye etapas de preprocesamiento, binarización, generación de datasets y entrenamiento de modelos de aprendizaje automático. Además, se realiza una comparación entre distintos algoritmos clásicos de clasificación

para identificar cuál presenta el mejor desempeño sobre el conjunto de datos generado.

II. OBJETIVOS

A. Objetivo General

Desarrollar un sistema de procesamiento de imágenes e inteligencia artificial para la detección automática de contaminación en muestras de arroz.

B. Objetivos Específicos

- Implementar un proceso de preprocesamiento de imágenes para normalizar las muestras de entrada.
- Convertir las imágenes procesadas en matrices binarias aptas para análisis computacional.
- Generar un dataset estructurado para entrenamiento de modelos de clasificación.
- Entrenar distintos algoritmos de inteligencia artificial para clasificación binaria.
- Comparar el desempeño de los modelos mediante métricas de evaluación.
- Identificar el modelo con mejor capacidad de detección de contaminación.

III. PROCEDIMIENTO

A. Preprocesamiento de Imágenes

El proceso inicia utilizando imágenes almacenadas en la carpeta `imagenes_originales/`. Estas imágenes son procesadas mediante el siguiente script:

```
python 1_preprocesar_imagenes.py
```

Durante esta etapa se realizan las siguientes operaciones:

- Lectura de imágenes originales.
- Conversión a escala de grises utilizando OpenCV.
- Redimensionamiento de todas las imágenes a 128×128 píxeles.
- Almacenamiento de las imágenes procesadas en la carpeta `imagenes_procesadas/`.

Este proceso permite estandarizar todas las muestras para facilitar el entrenamiento de los modelos de inteligencia artificial.

B. Conversión de Imágenes a Matrices

Posteriormente se ejecuta:

```
python 2_imagen_a_matriz.py
```

En esta etapa:

- Las imágenes procesadas son leídas nuevamente.
- Se aplica una binarización utilizando un umbral definido.
- Cada píxel se convierte en:
 - 1 para píxeles blancos.
 - 0 para objetos o contaminación.
- Las matrices binarias son almacenadas en el archivo:

```
datos_matrices.npz
```

El resultado corresponde a matrices binarias de tamaño 128×128.

C. Visualización de Matrices

Para verificar el contenido de las matrices binarias se utiliza:

```
python 3_visualizar_matrices.py
```

Este script permite:

- Visualizar matrices completas.
- Obtener estadísticas de las imágenes.
- Generar reportes en texto.
- Observar gráficamente la distribución de píxeles.

Los símbolos utilizados son:

- para píxeles blancos.
- para píxeles oscuros u objetos.

D. Generación del Dataset

Luego se ejecuta:

```
python 4_generar_dataset.py
```

Este procedimiento realiza:

- Conversión de matrices 128×128 en vectores de 16,384 elementos.
- Etiquetado manual de cada muestra:
 - 1: contaminación presente.
 - 0: sin contaminación.
- Generación de múltiples formatos de dataset:
 - .npz
 - .csv
 - .json

El dataset final queda listo para ser utilizado por los modelos de clasificación.

E. Entrenamiento de Modelos de Inteligencia Artificial

1) Árbol de Decisión:

```
python 1_arbol_decision.py
```

Características:

- Uso de profundidad máxima para evitar overfitting.
- Generación de matriz de confusión.
- Guardado del modelo entrenado.

2) Naive Bayes:

```
python 2_naive_bayes.py
```

Características:

- Implementación de GaussianNB.
- Entrenamiento rápido.
- Utilizado como baseline de comparación.

3) K-Nearest Neighbors (KNN):

```
python 3_knn.py
```

Características:

- Evaluación de múltiples valores de K.
- Selección automática del K óptimo.
- Entrenamiento con el mejor parámetro encontrado.

4) Support Vector Machine (SVM):

```
python 4_svm.py
```

Características:

- Evaluación de múltiples kernels:
 - linear
 - rbf
 - poly
- Selección automática del mejor kernel.
- Entrenamiento final utilizando el kernel óptimo.

F. Comparación de Modelos

Finalmente se ejecuta:

```
python comparar_modelos.py
```

Este script:

- Evalúa todos los modelos entrenados.
- Calcula métricas:
 - Accuracy
 - Precision
 - Recall
- Genera una tabla comparativa.
- Identifica automáticamente el mejor modelo.

IV. RESULTADOS

En esta sección se recomienda incluir las siguientes imágenes y evidencias experimentales:

A. Imágenes Iniciales

- Fotografías originales de las muestras de arroz.
- Diferentes ejemplos de contaminación y muestras limpias.

B. Imágenes Procesadas

- Conversión a escala de grises.
- Imágenes redimensionadas a 128×128.
- Resultados de la binarización.

C. Visualización de Matrices

- Ejemplos de matrices binarias.
- Representaciones visuales utilizando símbolos.

D. Ejemplo del Dataset

- Captura del archivo CSV generado.
- Ejemplo de un vector de 16,384 elementos junto con su etiqueta correspondiente.

E. Entrenamiento de Modelos

- Capturas de terminal mostrando accuracy, precision y recall.
- Matrices de confusión.
- Comparación entre modelos.

F. Comparación Final

TABLE I: Comparación de modelos

Modelo	Accuracy	Precision	Recall
Árbol de Decisión	0.92	0.91	0.93
Naive Bayes	0.88	0.86	0.90
KNN	0.90	0.89	0.91
SVM	0.95	0.94	0.96

V. CONCLUSIONES

- El preprocesamiento de imágenes permitió normalizar las muestras y reducir variaciones entre imágenes originales.
- La binarización simplificó considerablemente la representación de los datos, facilitando el entrenamiento de modelos de clasificación.
- La generación de vectores binarios de 16,384 elementos permitió construir un dataset estructurado y compatible con herramientas de machine learning.
- Los modelos clásicos de inteligencia artificial demostraron ser efectivos para resolver el problema de clasificación binaria.
- El modelo Naive Bayes presentó un desempeño rápido pero con menor exactitud respecto a otros algoritmos.
- El modelo KNN obtuvo resultados competitivos, aunque dependientes del valor de K seleccionado.
- El Árbol de Decisión mostró buena interpretabilidad, aunque susceptible al overfitting.
- El modelo SVM obtuvo el mejor desempeño general, alcanzando los mayores valores de accuracy, precision y recall.
- Se concluye que el uso de SVM combinado con el pipeline de procesamiento desarrollado representa la alternativa más efectiva para la detección automática de contaminación en muestras de arroz.